

Distributed Systems

Complete Technical Notes • App Scaling • DB Scaling • Sharding • Partitioning • Consistent Hashing

1. Why Multiple App Instances?

A single NestJS (or any Node.js) application is bound by a single event loop on a single thread. This creates three hard ceilings: CPU (one core), RAM (one heap), and network I/O of one machine. Running multiple instances behind a load balancer breaks all three ceilings.

Reason	What it solves
Fault Tolerance	One instance dying doesn't kill the service — traffic reroutes to healthy ones automatically.
Horizontal Scaling	5 instances = ~5x throughput. No single box matches this cheaply.
Zero-Downtime Deploy	Rolling update: kill one, deploy new, health-check, repeat. Users see no outage.
Geographic Distribution	Run instances in us-east, eu-west, ap-south — route users to the nearest.

Design Rule: Stateless Apps

Multiple instances expose bad in-memory assumptions. The fix is to keep the app stateless — all shared state lives outside (Redis, DB, message queue).

Anti-pattern	Problem	Fix
In-memory sessions (Map)	Each instance has its own memory — sessions don't exist in Redis	Store sessions in Redis
WebSockets / SSE	Client on instance A won't receive events if instance B publishes	Use Redis Pub/Sub or a message queue
Cron jobs	All 5 instances run the job → 5x duplicate side effects	Use a distributed lock or dedicated cron
Local file uploads	Instance A saves file, instance B can't find it	Use S3 or shared object storage

2. Traffic Spikes and the Event Loop

Node.js processes I/O-bound work well because it doesn't block while waiting. But the event loop itself is a single queue. A traffic spike overflows that queue.

Normal: [req1] → [req2] → [req3] processed quickly
Spike: [req1][req2]...[req500] queue builds → latency explodes
Even at 50ms/req, 500 concurrent = 25s wait time in queue

Resource	What degrades under spike
----------	---------------------------

Event loop	Saturated — response times go from ms to seconds
RAM / Heap	Each pending request holds memory → GC thrashing
DB connection pool	All connections in use → requests hang waiting
CPU (if compute work)	Blocks the loop for all other requests

Multiple instances fix this by distributing the spike across N event loops instead of hammering one.

3. Measuring Instance Capacity

Capacity = max RPS an instance can handle before your p99 latency SLO is breached. You must measure this, not guess.

Step 1 — Load Test

Use autocannon, k6, or Artillery. Ramp up concurrent connections until latency degrades or errors appear.

```
autocannon({ url: 'http://localhost:3000/api/orders', connections: 100, // concurrent users
duration: 30 }) // seconds
```

Step 2 — Key Metrics to Watch

Metric	Healthy	At Capacity
RPS	Climbing steadily	Plateauing
p99 latency	< 300ms	> 2000ms (SLO breached)
CPU	< 70%	> 90%
Event loop lag	< 10ms	> 500ms
Heap memory	Stable	Climbing / GC thrashing

Step 3 — Define Your SLO First

Pick a threshold before testing, e.g. 'p99 must stay under 300ms'. Capacity = max RPS before that threshold is crossed. The curve has a 'knee' — latency is flat, then suddenly explodes. That knee is your limit.

Rule of thumb: Run at 60-70% of measured capacity in production. Headroom absorbs spikes while auto-scaling kicks in (~90s lag).

4. Auto-Scaling with Kubernetes HPA

HPA (Horizontal Pod Autoscaler) watches metrics and adjusts replica count automatically. It runs a control loop every 15 seconds.

The Formula

$\text{desiredReplicas} = \text{ceil}(\text{currentReplicas} \times (\text{currentMetric} / \text{targetMetric}))$ Example: 7 pods running, CPU at 85%, target 70% $\text{ceil}(7 \times (85/70)) = \text{ceil}(8.5) = 9$ pods → spin up 2

Full Timeline During a Spike

t=0s → Spike arrives, 7 pods running fine t=15s → HPA checks: CPU 85% > target 70% → orders 9 pods t=45s → New pods Pending (k8s scheduling) t=60s → Pods Running but not Ready (app booting) t=90s → Health check passes → pods added to load balancer CPU drops to ~65%, system stabilised

There is a ~90 second lag between spike and relief. Your 60-70% headroom buys this time before things degrade.

Scale Down is Intentionally Slow

Default stabilisation window is 5 minutes. This prevents thrashing — constantly spinning up and killing pods during temporary dips.

HPA Config (Key Fields)

```
spec: minReplicas: 3 # never go below this maxReplicas: 20 # never go above this metrics: -
  type: Resource resource: name: cpu target: averageUtilization: 70 behavior: scaleUp:
  stabilizationWindowSeconds: 0 # scale up fast scaleDown: stabilizationWindowSeconds: 300 #
  scale down slowly
```

What HPA Can Scale On

Metric	Notes
CPU	Most common. Works well for compute-bound apps.
Memory	Less ideal — memory doesn't release cleanly.
RPS (custom metric)	Better for I/O-heavy Node apps. CPU may not spike even when overwhelmed.
Queue depth (external)	Great for consumer workers — scale based on Kafka/SQS lag.

For predictable spikes (9am traffic, sale events), use **Scheduled Scaling** — pre-scale before the spike arrives instead of reacting to it.

5. Handling 1000 RPS — Full Architecture

Step 1: Load test one instance → capacity = 200 RPS. Therefore: $1000 / 200 = 5$ instances minimum. Add 30% headroom = 7 instances.

Users (1000 RPS) | Load Balancer (Nginx / AWS ALB) | distributes evenly (~143 RPS each)
[N1][N2][N3][N4][N5][N6][N7] ← 7 NestJS instances | | PgBouncer Redis Cache | | Primary DB +
hot data in memory 2 Read Replicas

What Each Layer Does

Layer	Role	Impact
Load Balancer	Distributes 1k RPS across 7 instances	Each instance sees ~143 RPS (well w
Redis Cache	Serves ~70% of reads in <1ms	Only ~300 RPS actually reaches the D
PgBouncer	Pools 7x20=140 app connections into 20-DB max connections	DB never sees connection explosion
Read Replicas	Distributes read queries from 300 RPS	Primary only handles writes
K8s HPA	Adds pods when CPU > 70%	Survives 3x spikes automatically

Request Decision Flow

New request arrives | Is it cacheable? → YES → Redis → return (DB never touched) | NO Is it a read? → YES → Read Replica | NO Write → Primary DB via PgBouncer

6. DB Connections, Pooling, and PgBouncer

A database connection is a session between the app and PostgreSQL. Creating a connection is expensive — it consumes memory and CPU. So applications maintain a fixed pool of reusable connections.

The Formula

Total connections = instances x pool_size Example: 3 instances x pool_size 20 = 60 connections
Postgres default max_connections = 100 ~10 reserved for internals → ~90 usable 60 connections is fine here, but 5 instances x 20 = 100 → danger zone

How Many Connections Do You Actually Need?

connections ~= RPS x queries_per_request x query_time_seconds Example: 100 RPS, 5 queries/request, 100ms per query = 100 x 5 x 0.1 = 50 connections At 1000 RPS with Redis absorbing 70%: 300 RPS x 2 queries x 0.05s = 30 connections

More connections does NOT mean better performance. Beyond a point, each connection adds CPU overhead (context switching) and the DB slows down.

PgBouncer

PgBouncer sits between the app and the DB. It multiplexes thousands of incoming app connections into a small pool of real DB connections.

Without PgBouncer: 140 app connections → DB handles 140 connections With PgBouncer: 140 app connections → PgBouncer → 20-30 real DB connections

DB Breakdown Stages Under Load

Stage	Symptom	Root Cause	Fix
1	too many connections error	Pool exhausted	PgBouncer
2	Queries suddenly slow	Lock contention on hot rows	Optimise write patterns, sharding
3	DB CPU 100%	Missing indexes, N+1, bad joins	Index + query optimisation
4	Read replica lag	Write volume too high for replica	QRS + write queue
5	Random timeouts	Disk IO saturation	Vertical scale or shard

7. Table Partitioning

Partitioning splits one large table into smaller physical pieces within the same database. Your app code and queries don't change — Postgres routes automatically.

Partitioning vs Sharding

	Partitioning	Sharding
Scope	One DB, multiple physical chunks	Multiple separate DBs
App change needed?	No — Postgres handles routing	Yes — shard router required
Complexity	Low	High
Use when	Table has 100M+ rows, queries slow	Write load exceeds one DB's cap

How It Works

orders table partitioned by year: orders_2022 → 100M rows orders_2023 → 150M rows orders_2024 → 250M rows
SELECT * FROM orders WHERE date = '2024-01-01' → Postgres applies partition pruning → Scans only orders_2024 (not all 500M rows)

Partition Types

Type	Split By	Best For
Range	Date or numeric ranges	Time-series data (orders by month/year)
List	Specific values	Data by country, region, or status
Hash	hash(column) % N	Even distribution when no natural range

Try partitioning before sharding. It's the same DB, zero app changes, and often solves slow query problems on large tables entirely.

8. Database Sharding

Sharding splits data across multiple independent databases. Each shard has its own CPU, RAM, and disk. Use it when write load exceeds what one DB can handle, even after partitioning, caching, and read replicas.

The Shard Key — Most Critical Decision

The field you split data on. A good shard key distributes data evenly. A bad one causes a hot shard — one DB overloaded while others are idle.

Shard Key	Good For	Risk
user_id	User-centric apps	Celebrity users → hot shard
hash(user_id)	Even distribution	Range queries become impossible
geography	Regional apps	Uneven region sizes
tenant_id	Multi-tenant SaaS	Large tenants → hot shard

Types of Sharding

Type	How	Pros	Cons
Range	user_id 1-1M → Shard 1, 1M-2M → Shard 2	Range queries easy	New data always in one shard
Hash	hash(user_id) % N → Shard N	Even distribution	Range queries hard
Directory	Lookup table maps keys to shards	Full flexibility	Lookup table is a bottleneck

Shard Router

A new critical component between app and DB. It calculates which shard to query for every request. Can live in app logic, middleware, or a dedicated proxy (Vitess for MySQL, Citus for Postgres).

Request: GET /orders?user_id=1500 | Shard Router hash(1500) % 3 = 2 | Shard 2 ← only this DB is queried

Hard Problems Sharding Introduces

Problem	Why it's hard	Mitigation
Cross-shard queries	No shard key → must scatter to all shards and aggregate results	Design queries to always include shard key

Cross-shard transactions	Shard A commits, Shard B fails → inconsistent data	2-phase commit or saga pattern
Resharding	Adding a new shard remaps almost all data when using hash % N	Consistent Hashing

When to Shard — Exhaust These First

1. Add indexes ← fixes 80% of slow query problems 2. Optimise queries ← eliminate N+1, fix bad joins 3. Read replicas ← offload read traffic 4. Redis caching ← eliminate repeated reads entirely 5. Vertical scaling ← buy time cheaply 6. Table partitioning ← split large tables within same DB ↓ still not enough? 7. Shard

9. Consistent Hashing — Solving the Resharding Problem

The Problem with hash % N

3 shards: $\text{hash}(\text{user_id}) \% 3$ 4 shards: $\text{hash}(\text{user_id}) \% 4$ Result: Almost ALL existing users map to different shards → Massive data migration while serving live traffic ■

The Consistent Hashing Ring

Instead of hash % N, imagine a ring of positions (0 to 360 degrees). Each shard owns a range on the ring. A user's hash maps them to a point; they go to the nearest shard clockwise.

Before (3 shards on ring): Shard 1 owns $0^\circ - 120^\circ$ Shard 2 owns $120^\circ - 240^\circ$ Shard 3 owns $240^\circ - 360^\circ$ After adding Shard 4 at 60° : Shard 1 owns $0^\circ - 60^\circ$ ← reduced Shard 4 owns $60^\circ - 120^\circ$ ← new, takes only Shard 1's slice Shard 2 owns $120^\circ - 240^\circ$ ← untouched Shard 3 owns $240^\circ - 360^\circ$ ← untouched Only 1/N of data moves. hash % N moves nearly ALL data.

Migration Strategy with Consistent Hashing

Phase 1 – Dual Write New writes → Shard 1 AND Shard 4 (both stay in sync) Reads → still from Shard 1 Phase 2 – Background Migration Copy Shard 1's affected data to Shard 4 Router tracks which users are migrated Phase 3 – Cutover Migrated users → reads now from Shard 4 Stop dual write Phase 4 – Cleanup Delete migrated data from Shard 1

Shard Router During Migration

Request: user 1500 | Shard Router checks migration state | Is user 1500 migrated yet? NO → read from OLD shard (Shard 1) YES → read from NEW shard (Shard 4)

Consistent hashing is used under the hood in Cassandra, Redis Cluster, and DynamoDB for exactly this reason — adding nodes moves only 1/N of data.

10. Quick Reference — When to Use What

Problem	Solution	Complexity
Single instance crashes	Multiple instances + load balancer	Low
Traffic spike overwhelms 1 instance	HPA auto-scaling + 60-70% headroom	Medium
Too many DB connections	PgBouncer connection pooler	Low
Repeated reads hammering DB	Redis caching layer	Low-Medium
Read queries slow under load	Read replicas	Medium
One large table queries slow	Table partitioning (same DB)	Low

Write load too high for 1 DB	Sharding across multiple DBs	High
Adding new shard breaks routing	Consistent hashing	High
Known spike (e.g. sale event)	Scheduled pre-scaling	Low

These notes cover: Multiple App Instances · Traffic Spikes · Capacity Measurement · K8s HPA · 1k RPS Architecture · DB Connections · PgBouncer · Table Partitioning · Sharding · Consistent Hashing

Important Note: Cross-Shard Joins (Must Know)

In a sharded database system, performing joins across multiple shards is highly inefficient and often not supported. Each shard operates as an independent database, so joining data that resides on different shards requires fetching data from multiple nodes and combining it at the application or coordination layer, which leads to high latency and complexity.

To avoid this problem, systems are designed such that related data is stored in the same shard by choosing an appropriate shard key. For example, if complaints are frequently joined with users, both should be sharded using the same key like `user_id` so they are co-located.

Another common approach is denormalization, where related data is stored together to eliminate the need for joins. In CQRS architectures, this is typically handled by maintaining a read-optimized datastore where data is already aggregated.

Key takeaway: If your system requires joins, ensure the data resides in the same shard. Cross-shard joins should be avoided in distributed systems.